

Sim-to-Real RL: Tutorial from NVIDIA Isaac Sim to Gazebo and Real ROS 2 Robots

Sahar Salimpour, Jorge Peña-Queralta

Abstract—Unprecedented agility and dexterous manipulation have been demonstrated with controllers based on deep reinforcement learning (RL), with a significant impact on legged and humanoid robots. Modern tooling and simulation platforms, such as NVIDIA Isaac Sim, have been enabling such advances. This tutorial focuses on demonstrating the applications of Isaac in local planning and obstacle avoidance as one of the most fundamental ways in which a mobile robot interacts with its environments. The tutorial aims to provide a base framework for end-to-end local navigation policies and how a custom robot can be trained in such simulation environment. In the original paper that this tutorial supports [1], we benchmark end-to-end policies with the state-of-the-art Nav2, navigation stack in Robot Operating System (ROS). We also cover the sim-to-real transfer process by demonstrating zero-shot transferability of policies trained in the Isaac simulator to real-world robots. This is further evidenced by the tests with different simulated robots, which show the generalization of the learned policy. Finally, the benchmarks demonstrate comparable performance to Nav2, opening the door to quick deployment of state-of-the-art end-to-end local planners for custom robot platforms, but importantly furthering the possibilities by expanding the state and action spaces or task definitions for more complex missions. Overall, with this article we introduce the most important steps, and aspects to consider, in deploying RL policies for local path planning and obstacle avoidance with Isaac Sim training, Gazebo testing, and ROS 2 for real-time inference in real robots. The reproducible code is available at <https://github.com/sahars93/RL-Navigation>.

Index Terms—Reinforcement learning (RL); Deep reinforcement learning; Sim-to-real transfer; Mobile robotics; Local planning; Obstacle avoidance; Gazebo; ROS 2; Nav2; End-to-end control.



This document acts as a step-by-step tutorial and appendix to support the paper *Sim-to-Real Reinforcement Learning for Local Mobile Robot Navigation in Dynamic Environments*¹. The overall idea is illustrated in Fig. 1.

1 RL WITH ISAAC SIM

The introduction of the NVIDIA Omniverse Isaac Gym and Orbit frameworks has arguably aided in widening of audience and applications of deep reinforcement learning research. These advancements make RL-based training for real-world robotics more accessible, replacing the need for dedicated hardware or software frameworks.

1.1 Isaac Sim

Isaac Sim, a GPU-based general-purpose physics simulation platform from Nvidia, serves as an

extensible robotics simulator that empowers designers, researchers, and developers to create, test, and train AI-based robots such as wheeled robots, legged robots, and drones. Leveraging the power of NVIDIA Omniverse, Isaac Sim provides scalable, photorealistic, and physically accurate virtual environments for high-fidelity simulations. It can simulate realistic sensor models such as camera, lidar, and IMU, and a variety of objects and scenes, enabling tasks such as manipulation, navigation, synthetic data generation, and various computer vision applications through Python, ROS integration, and Isaac SDK.

1.2 RL in Isaac Sim / Gym

Omniverse Isaac Gym is an extension for reinforcement learning in robotics which is built on top of NVIDIA Isaac Sim. Isaac Gym is highly parallelized simulations by conducting both physics simulation and policy training on the GPU through an API,

1. Full paper: <https://arxiv.org/abs/2501.02902>

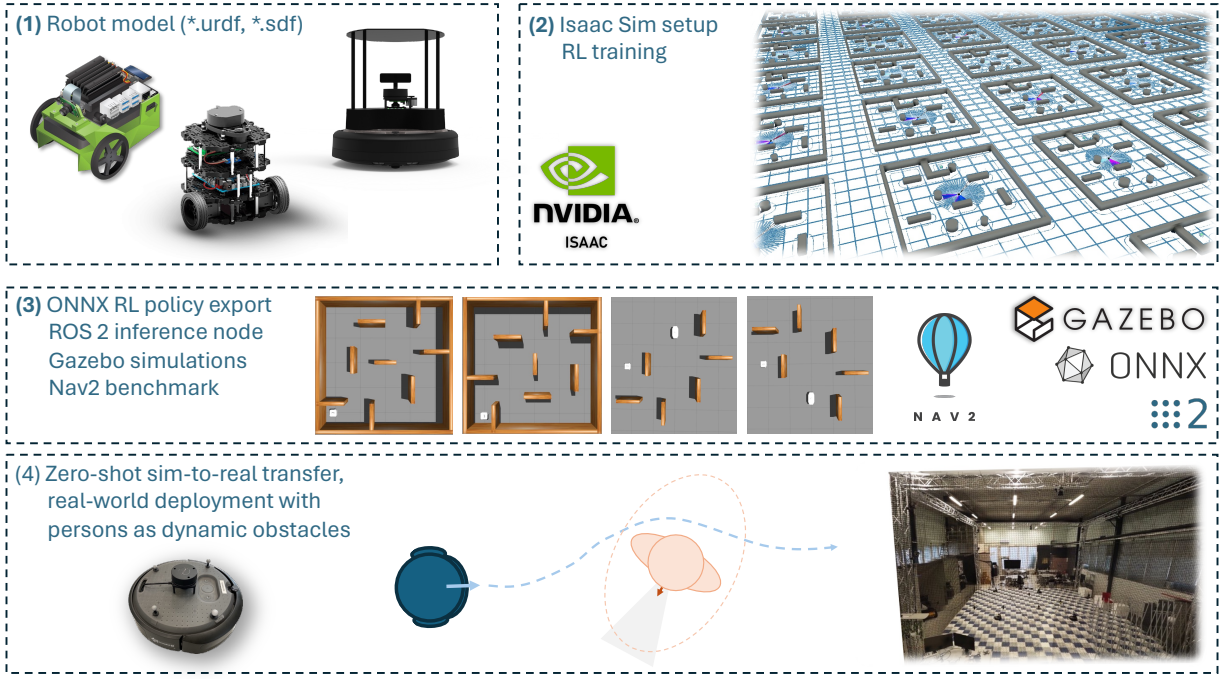


Fig. 1: Conceptual illustration of the sim-to-real workflow described in [1].

based on the vectorization of observations and actions. This framework offers a straightforward interface for training RL agents and supports various RL algorithms. In the latest releases of Isaac Gym, RL Games is introduced as the default library for running example environments. Whether it is training robotic agents to perform complex tasks, fine-tuning and optimizing RL policies, or evaluating their performance, Isaac Gym provides a bridge between the simulation environment and RL algorithms.

More recent RL framework, also powered by NVIDIA Isaac Sim, are Orbit and Isaac Lab. Orbit offers a comprehensive suite of features, including support for various robot platforms, sensors, teleoperation, imitation Learning, and motion planning across diverse robotic applications, while Isaac Lab provides more comprehensive environments for different types of robots and tasks². In these new frameworks, beginners may encounter a steeper learning curve. This article, therefore, focuses on the more documented and tested Isaac Gym as the first tutorial for straightforward path planning and navigation implementation, which can be further developed in Isaac Lab for more advanced features.

2 RL WORKFLOW

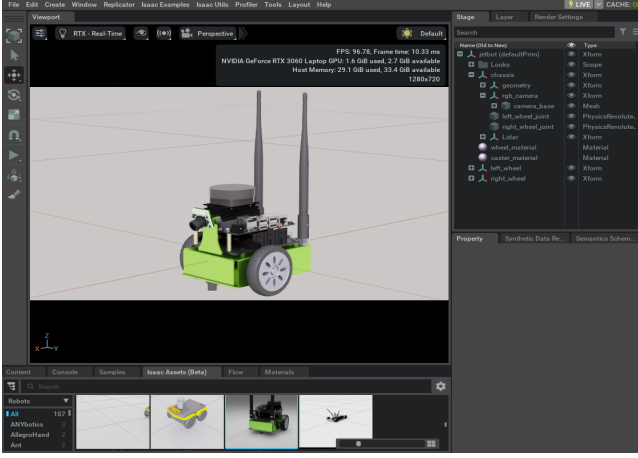
In Isaac Gym, the development and simulation of a customized RL agent navigation task needs a

few basic steps for effective simulation, training, and testing. This includes defining the simulation environment and robot, crafting a Python script for the task class to specify goals, reward computation, and reset management, and utilizing two YAML configuration files—one for task parameters and another for training parameters—to complete the task. This section provides an overview of these essential core components.

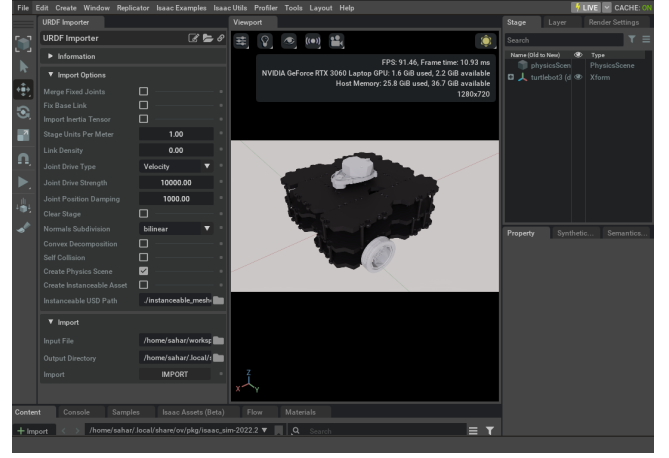
2.1 Robot And Environments

Isaac Gym offers a user-friendly API for creating and configuring scenes with custom robots and objects. Many ROS users utilize the Unified Robot Description Format (URDF), a popular format for describing the basic robot cell and geometry, and practical applications. Isaac Sim supports various file formats, including URDF, Multi-Joint dynamics with Contact (MJCF), and Universal Scene Description (USD). It is feasible to incorporate custom robots from URDF and MJCF files into tasks or convert them to USD format using the Isaac Sim Importer extensions. Two robot models were employed in our mobile robot navigation task, the modified Isaac built-in Jetbot robot with Lidar sensor and the USD model of the Turtlebot3-Waffle from the TurtleBot3 Simulation ROS Package, converted through the Isaac URDF Importer for additional experiments.

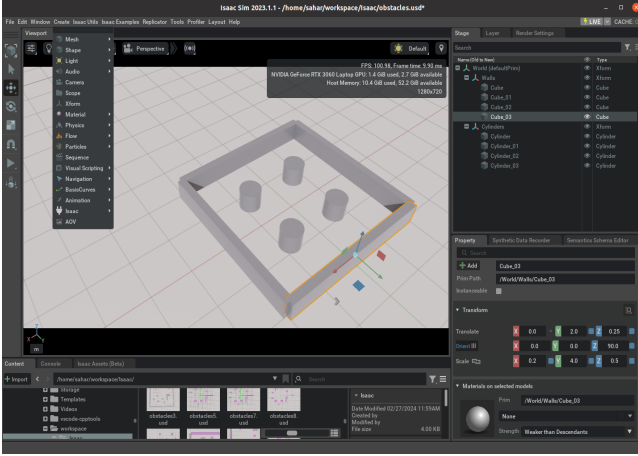
2. <https://isaac-sim.github.io/IsaacLab/main/index.html>



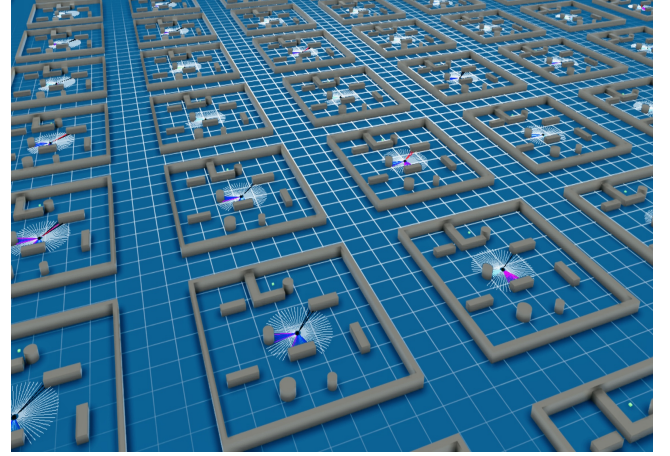
(a) Isaac Jetbot robot



(b) URDF Importer Extension



(c) A custom environment in Isaac Sim



(d) Isaac Gym

Fig. 2: Robots and Environments

2.2 Task

A variety of reinforcement learning tasks are provided at Omniverse Isaac Gym extension, where main functionalities such as performing episode resets, applying actions, collecting observations, and computing rewards are implemented in this task class. Our *JetbotTask*, inherits from the *BaseTask* class in *omni.isaac.core*, comprises several key components. The general structure of the definition of each component of a new RL task is shown in Listing 1. The Initialization phase, the *init* function, sets initial configurations for the environment. The initial setup for each task is detailed in its dedicated task YAML file. An overview of this file, along with a few sample parameters, is presented in ???. One can specify parameters related to the environment and simulation within this file. These include the number of environments, various sensors and USD configurations for the robot and objects, specifying CPU or GPU pipeline, and applying noise to obser-

vations, actions, and other properties in the domain randomization part.

Action and observation space and other parameters such as the episode’s length are also defined in the *init* function. The *set_up_scene* function is for setting up the scene by creating *ArticulationView* or *RigidPrimView* objects. This function involves defining the scene, sensor, and robot, or loading assets from USD, URDF, and MJCF file formats. The *get_observation* function generates the observation space using Lidar ranges, information on the target’s relative position, and the robot’s state. Computations required before stepping into the physics simulation, such as applying actions to move the robot based on policy decisions or resetting the environment, occur in the *pre_physics_step* function. The calculation of rewards, resets, and extra buffers is handled in the *calculate_metrics* function. Finally, determining which environments need resetting is done in the last function of the task. Besides

```

class NewTask (RLTask):
    def __init__(
        config # Defined in the task.yaml
    )
    def set_up_scene(self, scene):
        self.get_robot()
        self._robots = RobotView()
        scene.add(self._robots)
        return
    def get_robot():
        robot = Robot()
        add_reference_to_stage()
    def get_observations(self):
        self.obs_buf[:] = obs
        return observations
    def pre_physics_step(self, actions):
        self.reset_idx(reset_env_ids)
        actions = actions.to(self._device)
    def calculate_metrics(self):
        ...
    def is_done(self):
        ...

```

Listing 1: General structure of a new RL task definition in OmniIsaacGym.

```

name: Robot
env:
  numEnvs:
sim:
  use_gpu_pipeline: ...
  use_flatcache: ...
  PhysX:
    use_gpu: ...
  Robot:
    ...
  object:
    ...
  domain_randomization:
    observations:
      ...
    actions:
      ...

```

Listing 2: Core components of *task.yaml*.

the task config file, each task is accompanied by its configuration file containing training parameters such as the model and network structures, and the PPO parameters such as the learning rate, as shown in ???. These parameters are passed through *rlgames_train.py*.

2.3 From Isaac To Gazebo

Before deploying navigation policies in the real world, we evaluate the trained policy within the Gazebo simulation environment. Gazebo, a widely adopted simulator in the global robotics community, directly interfaces with the ROS through user-friendly packages. This integration facilitates the creation of accurate simulations, and the outcomes obtained can be directly implemented on the real robots with only ROS installed, regardless of their

```

params:
  algo:
    name: ...
  model:
    name: ...
  network:
    mlp:
      units: ...
      ...
    rnn:
      name: ...
      ...
  config:
    learning_rate: ...
    minibatch_size: ...
    mini_epochs: ...
    critic_coef: ...

```

Listing 3: Core components of *train.yaml*.

software architecture. There are many Gazebo simulation packages available, featuring different differential drive robots equipped with Lidar sensors.

In order to test in the Gazebo simulator, the trained model weights are exported in the format of ONNX model and integrated into a ROS node. The ONNX model can be imported and utilized for both simulation and real-world scenarios.

2.4 ROS 2 Node Deployment

A ROS2 node is defined to handle the ROS operations within the robots. Upon initialization, such as the target's position, it sets up subscriptions to topics such as the Scan topic to get the Lidar ranges and the robot's position through the Odometry topic, along with a publisher for *cmd_vel* commands. Additionally, the class integrates an ONNX model, for inference. Various callback functions are implemented, such as *Odometry_callback()*, and *scan_callback()*, to process incoming messages from subscribed topics. Finally, the *send_control()* function generates robot control commands based on the model's outputs and publishes them via *cmd_vel*.


```

class ROSNode(Node):
    def __init__(self):
        super().__init__('ROS_node')
        self.Pose_subscription = ...
        self.LidarRages_subscription = ...
        self.cmd_vel_publisher = ...
        self.ort_model =
            ort.InferenceSession("model.onnx")
    def Odometry_callback(self, msg):
        self.robot_position = ...
        self.robot_orientation = ...
    def scan_callback(self, msg):
        self.ranges = ...
    def send_control(self):
        observation = ...
        outputs = self.ort_model.run()
        self.cmd_vel_publisher.publish(twist)
def main():
    ...
if __name__ == '__main__':
    main()

```

Listing 4: ROS2 Node for real-time inference.

REFERENCES

- [1] S. Salimpour, J. Peña-Queralta, D. Paez-Granados, J. Heikkonen, and T. Westerlund, "Sim-to-real reinforcement learning for local mobile robot navigation in dynamic environments," *arXiv preprint arXiv:2501.02902*, 2025.
- [2] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, "From the desks of ros maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2," *Robotics and Autonomous Systems*, vol. 168, 2023.
- [3] W. Zhao, E.-A. Rantala, J. Pajarinen, and J. P. Queralta, "Less is more: Robust robot learning via partially observable multi-agent reinforcement learning," *arXiv*, 2023.
- [4] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *IEEE SSCI*, 2020.
- [5] S. Aradi, "Survey of deep reinforcement learning for motion planning of autonomous vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, 2020.
- [6] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, *et al.*, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, 2023.
- [7] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, "Learning robust autonomous navigation and locomotion for wheeled-legged robots," *Science Robotics*, vol. 9, no. 89, 2024.
- [8] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, vol. 8, no. 82, 2023.
- [9] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, 2024.
- [10] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "Rloc: Terrain-aware legged locomotion using reinforcement learning and optimal control," *IEEE Transactions on Robotics*, vol. 38, no. 5, 2022.
- [11] W. Li, M. Yue, J. Shangguan, and Y. Jin, "Navigation of mobile robots based on deep reinforcement learning: Reward function optimization and knowledge transfer," *Intl. Journal of Control, Automation and Systems*, 2023.
- [12] H. Surmann, C. Jestel, R. Marchel, F. Musberg, H. Elhadj, and M. Ardani, "Deep reinforcement learning for real autonomous mobile robot navigation in indoor environments," *arXiv*, 2020.
- [13] N. Ü. Akmandor, H. Li, G. Lvov, E. Dusel, and T. Padir, "Deep reinforcement learning based robot navigation in dynamic environments using occupancy values of motion primitives," in *IEEE IROS*, 2022.
- [14] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, 2023.